

Vehicle Tracking and Counting System

Internship Project Report | Computer Vision with YOLOv8

1. Project Overview

This project implements a real-time vehicle tracking and counting system using computer vision. The system processes traffic video footage, detects vehicles (cars, buses, trucks, and motorcycles) using a YOLOv8 object detection model, assigns a persistent unique ID to each vehicle across frames, and counts vehicles as they cross a defined line in the video, producing a live count broken down by vehicle class.

2. Objectives

- Detect vehicles in traffic video footage using a pretrained deep learning model.
- Track each detected vehicle across frames, assigning it a consistent, unique identity.
- Count vehicles as they cross a virtual line, avoiding duplicate counts of the same vehicle.
- Display live visual feedback: bounding boxes, vehicle IDs, counting line, and running totals.
- Produce a class-wise breakdown of vehicle counts (cars, buses, trucks, motorcycles).

3. Tools and Technologies

Component	Technology Used
Programming Language	Python 3.14
Object Detection Model	YOLOv8n (Ultralytics)
Computer Vision Library	OpenCV (opencv-python)
Numerical Computation	NumPy
Development Environment	Visual Studio Code
Video Source	Sample traffic surveillance footage (traffic.mp4)

4. System Architecture

The system operates in a continuous frame-by-frame pipeline:

- 1 Frame Capture** — Each frame is read from the video file using OpenCV's VideoCapture.

- 2 **Detection** — The frame is passed to a YOLOv8n model, which returns bounding boxes, class labels, and confidence scores for all detected objects.
- 3 **Filtering** — Detections are filtered to keep only vehicle classes: car, bus, truck, and motorcycle.
- 4 **Tracking** — Each vehicle detection's center point is passed to a custom centroid-based tracker, which matches it against vehicles tracked in the previous frame and assigns a consistent ID.
- 5 **Counting** — When a confirmed, tracked vehicle's center crosses a virtual horizontal line, it is counted exactly once.
- 6 **Visualization** — Bounding boxes, vehicle IDs, the counting line, and running totals are drawn onto the frame and displayed in real time.

5. Key Technical Decision: Custom Tracker vs. ByteTrack

Ultralytics' built-in **model.track()** function relies on ByteTrack/BoT-SORT, which in turn depends on the **lap** Python package for solving the linear assignment problem during tracking. During development, this dependency could not be installed in the project environment for two reasons:

- The Python interpreter (3.14) is very new, and no prebuilt **lap** wheel was available for it, forcing a build-from-source attempt that failed.
- The Python installation directory (Program Files) required administrator privileges to write to, which was not available, resulting in an "Access is denied" error during installation.

Resolution: Rather than blocking progress on an environment dependency, a lightweight **custom centroid-based tracker** was implemented from scratch using only NumPy — no external tracking library was required. This tracker matches each detected vehicle in the current frame to the closest tracked vehicle from the previous frame (based on center-point distance), assigns new IDs to unmatched detections, and removes tracks that go unmatched for too long. A *minimum-hit-count confirmation rule* was also added: a track must be observed for several consecutive frames before it is eligible to be counted, which filters out noisy, one-frame detection flicker that would otherwise inflate the count.

This approach demonstrates the ability to diagnose an environment/dependency issue, evaluate trade-offs, and engineer a practical, working alternative under real constraints — a common scenario in production software and data science environments.

6. Tracker Configuration

Parameter	Value	Purpose
max_distance	45 px	Maximum center-point movement allowed between frames for a match
max_missed	20 frames	How long a track survives without a matching detection before being dropped
min_hits	3 frames	Minimum consecutive detections before a track is "confirmed" and countable

7. Results

The system was tested on an 18-second aerial highway traffic clip containing dense, multi-lane, bidirectional traffic. Detection, tracking, and counting ran successfully end-to-end with no errors.

Metric	Result
Total vehicles counted	31 (final tuned configuration)
Vehicle classes detected	Car, Truck (Bus and Motorcycle: none present in clip)
Processing outcome	Completed without runtime errors

Note: Earlier tracker configurations produced inflated counts (71–82) due to ID switching between closely spaced vehicles in adjacent lanes. Tuning `max_distance` downward and adding the `min_hits` confirmation rule reduced this significantly, bringing the count much closer to a realistic estimate.

8. Limitations and Future Improvements

- **Tracking accuracy:** The custom centroid tracker is simpler than ByteTrack/DeepSORT and can occasionally switch IDs between closely spaced vehicles in dense traffic. Using a proper Kalman-filter-based tracker (once the ``lap`` dependency issue is resolved, e.g. via a Python 3.11/3.12 virtual environment) would likely improve accuracy further.
- **Direction-aware counting:** The current system counts all crossings in one direction. Separate counters for each direction of travel could be added by comparing a vehicle's previous and current position relative to the line.
- **Speed estimation:** Vehicle speed could be estimated using frame rate and pixel-to-real-world distance calibration.
- **Model upgrade:** A larger YOLOv8 variant (e.g. YOLOv8s or YOLOv8m) could improve detection accuracy at the cost of slower processing speed.

9. Conclusion

This project successfully demonstrates an end-to-end computer vision pipeline for vehicle detection, tracking, and counting using YOLOv8 and OpenCV. When a key dependency (`lap`) proved incompatible with the development environment, a custom tracking solution was engineered as a practical workaround, highlighting adaptability and problem-solving under real-world constraints. The resulting system runs reliably and produces class-wise vehicle counts suitable for traffic monitoring use cases.